Real-Radio OFDM Prototype with Automated Data Collection and Neural Gate for Deep Learning-Aided Packet Detection

Kaikai Liu
Department of Electrical Engineering
San José State University
San José, CA 95192, USA
kaikai.liu@sjsu.edu

***Abstract*—We present AISensing, an end-to-end real-radio OFDM prototype built on two ADALM-PlutoSDR units connected via a coaxial RF link, with a custom Python PHY stack running on a Jetson Orin (transmitter) and an Intel-based workstation with an NVIDIA Titan V GPU (receiver). The system addresses a central challenge in applied deep learning for communications: acquiring large, accurately annotated datasets directly from real hardware without manual labelling. Our automated pipeline sweeps receiver gain, modulation order (QPSK and 16-QAM), and transmit power, logging every capture with CRC-verified ground-truth labels, physical-layer diagnostics, and per-window neural inference scores. A three-tier Mixture Logger separates captures into *hard negatives* (packet-like decoding failures), *mid-confidence* negatives, and *background* noise windows, saving the raw IQ windows, cross-correlation profiles, and full diagnostic metadata to compressed NPZ archives for offline deep learning research. To filter the expensive demodulation step, we train an online NEURALGATE — a 61-feature, 128-hidden-unit MLP — on the automatically labelled dataset. A critical lesson is documented: features computed after demodulation (EVM, CFO, SNR) must be *excluded* from features used at inference time, as their absence at gate time causes catastrophic distributional shift. The corrected NEURALGATE v2 achieves ROC-AUC = 0.946, and the subsequent hard-negative mined NEURALGATE v3 reaches ROC-AUC = 0.995 with precision = 0.984 and recall = 0.963. Closed-loop deployment of NEURALGATE v3 in a follow-up sweep (v4) achieves 96.9% QPSK decode rate and 52.3% QAM-16 decode rate, with the characteristic ADC-saturation pattern (8.1% at 60 dB, 65.6% at 40 dB) confirming gate-guided hard-negative mining operates correctly in the field. Our full dataset exceeds 36,000 annotated captures and 19,000 NPZ archives totalling over 30 GB, collected fully autonomously.**

***Index Terms*—Software-defined radio, OFDM, deep learning, neural gate, automated data collection, mixture logging, dataset annotation, PlutoSDR.**

I. INTRODUCTION

Deep learning for physical-layer communications has produced impressive simulation results [1], [2], yet deployment on real hardware exposes a persistent gap: models trained in simulation fail when channel statistics differ from the real RF environment, and models trained on recorded IQ data require expensive manual annotation or rely on imperfect channel emulators.

Closing this gap demands *self-supervised, hardware-in-the-loop data collection* — systems that run continuously, exploit existing PHY error-detection (CRC) for ground-truth labels, and log rich metadata that enables offline deep learning research without re-running experiments.

This paper makes the following contributions:

1) **Real-radio OFDM prototype.** A complete, packet-level OFDM physical layer running on commodity SDR hardware (ADALM-PlutoSDR) with Schmidl-Cox synchronisation [3], OFDM channel estimation, QPSK/16-QAM demodulation, and CRC-32 verification.
2) **Automated labelling pipeline.** A gain-sweep receiver that continuously annotates every capture with CRC-derived labels, MAD z-score energy features, normalised cross-correlation (NCC) scores, post-demodulation diagnostics (SNR, CFO, EVM), and neural gate scores, writing everything to structured CSV and NPZ files without human intervention.
3) **Three-tier Mixture Logger.** A configurable save policy that ensures *all three* data regimes — clean packets, hard decoding failures, and background noise — are collected in a balanced ratio, enabling contrastive and self-supervised training strategies.
4) **NEURALGATE with documented pitfall.** An online MLP gate that filters pre-demodulation. We document the *feature-leakage* pitfall — post-demod features inadvertently leak the label into the training features, causing trivial separation that vanishes at inference time — and present a corrected model that generalises reliably.
5) **Continuous improvement loop.** The system is designed so that each experiment automatically enriches the training set with new hard negatives, enabling iterative retraining without additional engineering effort.

Fig. 1: End-to-End Real-Radio AI Sensing Prototype

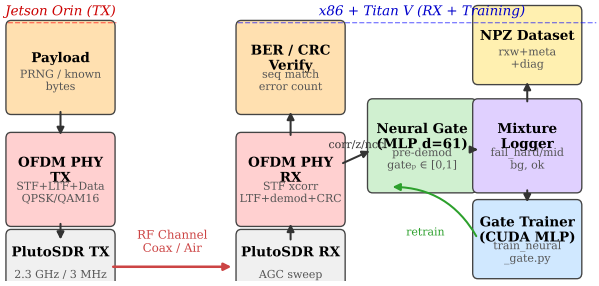


Fig. 1. End-to-end AISensing prototype. Jetson Orin transmits OFDM frames; the local workstation receives, annotates, gates, and stores captures autonomously. Dashed lines show the iterative retraining feedback loop.

TABLE I
OFDM PHY PARAMETERS

Parameter	Value
FFT size N_{FFT}	64
Cyclic prefix N_{CP}	16
Symbol length	80 samples
Data subcarriers N_{data}	48
Pilot subcarriers	$\{-21, -7, 7, 21\}$
STF (Schmidl-Cox)	6 half-period repetitions
LTF symbols	4
Modulations	QPSK, 16-QAM
Payload	512 bytes (PRNG, CRC-32)
Frame (QPSK)	44 OFDM symbols
Frame (QAM-16)	22 OFDM symbols
Centre frequency	2.3 GHz
Sample rate / BW	3 MHz / 3.6 MHz

II. System Design and Implementation

A. Hardware Platform

Figure 1 shows the end-to-end system. Two ADALM-PlutoSDR units operate at $f_c = 2.3\,\text{GHz}$ with $f_s = 3\,\text{MHz}$ sample rate. The transmitter runs on an NVIDIA Jetson Orin embedded platform; the receiver runs on an Intel 12th-generation workstation equipped with an NVIDIA Titan V (5120 CUDA cores, 12 GB HBM2). The two SDRs are connected via a low-loss SMA coaxial cable, eliminating propagation loss while preserving realistic hardware impairments: ADC quantisation, IQ imbalance, local-oscillator frequency offset (measured CFO $\approx$ 460–520 Hz), and non-linear amplifier behaviour at high gain settings.

B. Custom OFDM Physical Layer

Both TX and RX are implemented in Python with NumPy, with optional Numba JIT acceleration for the inner cross-correlation loop. Table I summarises the PHY parameters.

Transmitter. The transmitter builds a frame: a 4-byte magic word, 2-byte sequence number, 2-byte length, payload bytes, and CRC-32. The frame is modulated as OFDM symbols with pilot-based channel estimation reference symbols (LTF), wrapped in a Schmidl-Cox STF for timing synchronisation, and streamed continuously to the PlutoSDR DMA buffer.

Receiver. The receiver runs a ring-buffer architecture: an acquisition thread fills a circular buffer from the PlutoSDR, while a DSP thread processes overlapping windows of 262 144 samples at 65 536-sample hop. For each window the pipeline (Fig. 1, right side) executes the following stages.

1) **Energy gate.** A sliding 512-sample energy estimate $e[n]$ is computed. The MAD z-score $z_{\text{MAD}} = (e_{\max} - \tilde{e})/(1.4826 \cdot \text{MAD}(e))$ filters pure-noise windows: if $z_{\text{MAD}} < 8$ the window is skipped.
2) **Cross-correlation and NCC.** The window is cross-correlated with the STF reference. The top-k ($k = 8$) peaks are retained. For each peak the normalised cross-correlation score ρ_{NCC} is computed to rank candidates.
3) **NeuralGate inference.** A 61-feature MLP evaluates the window *before* demodulation and returns a gate score $g_p \in [0, 1]$. If $g_p < \tau$ the save decision may still proceed for hard-negative collection (see Section IV).
4) **Demodulation.** For candidates with $\rho_{\text{NCC}} \geq 0.15$, full OFDM demodulation is attempted: LTF-based least-squares channel estimation, pilot phase-tracking, and maximum-likelihood symbol detection.
5) **CRC verify.** The decoded frame is CRC-32 verified and compared against the known PRNG reference to compute BER.

C. Receiver Gain Sweep

A configurable gain-sweep manager steps the PlutoSDR RX VGA gain through a list (e.g. $[60, 55, 50, 45, 40, 35, 30, 25]$ dB) every 25 seconds, collecting approximately 150–200 captures per gain level. This produces a controlled SNR-versus-gain characterisation curve in a single unattended experiment.

III. Automated Data Collection and Annotation

A. Three-Tier Mixture Logger

A naïve policy that saves only successfully decoded packets yields a heavily class-imbalanced dataset biased toward high-SNR conditions. Conversely, saving every window is storage-prohibitive and fills the dataset with uninformative noise. Our *Mixture Logger* implements three tiers based on per-window signal quality indicators:

- `fail_hard` (always save): any of $z_{\text{MAD}} \geq 200$ or $\rho_{\text{NCC}} \geq 0.70$ or $g_p \geq 0.80$. These are confirmed packet-like windows where demodulation failed — the hardest negatives for the gate.
- `fail_mid` (saved with probability $p_{\text{mid}} = 0.5$): $z_{\text{MAD}} \geq 30$ or $\rho_{\text{NCC}} \geq 0.30$ or $g_p \geq 0.30$. Medium-confidence candidates that add diversity without over-sampling.
- `fail_bg` (saved with probability $p_{\text{bg}} = 0.05$): window passed the energy gate but below mid thresholds. Provides balanced background-noise negatives.

Successfully decoded packets are always saved (`ok`). This policy is summarised in Algorithm 1.

Algorithm 1 Mixture Logger Save Decision

if status = OK **then**
  return `ok`
elsif $z_{\text{MAD}} \geq z_H$ **or** $\rho_{\text{NCC}} \geq \rho_H$ **or** $g_p \geq g_H$ **then**
  return `fail_hard`
elsif $z_{\text{MAD}} \geq z_M$ **or** $\rho_{\text{NCC}} \geq \rho_M$ **or** $g_p \geq g_M$ **then**
  return `fail_mid` with prob. p_M
elsif $z_{\text{MAD}} \geq z_{\text{th}}$ **then**
  return `fail_bg` with prob. p_{bg}
else
  return `skip`
end if

TABLE II
DATASET SUMMARY ACROSS ALL EXPERIMENTS

Set	Mod	TX Gain	Caps	OK	OK%	NPZ
v1-run1	QPSK	0 dB	6180	5336	86.3	6192
v1-run2	QPSK	0 dB	1500	1280	85.3	1500
v1-run3	QAM-16	0 dB	1500	722	48.1	1500
v1-run4	QAM-16	0 dB	1500	715	47.7	1500
v2-run1	QPSK	−20 dB	1500	613	40.9	613
v2-run2	QAM-16	0 dB	1500	648	43.2	648
v2-run3	QPSK	−30 dB	1500	0	0.0	0
v2-run4	QPSK	−25 dB	1500	0	0.0	0
v3-run1	QPSK	0 dB	1500	1315	87.7	1500
v3-run2	QPSK	−20 dB (fade)	4500	0	0.0	232
v3-run3	QAM-16	0 dB	1500	556	37.1	1500
v4-run1	QPSK	0 dB	1500	1454	96.9	1500
v4-run2	QAM-16	0 dB	1500	784	52.3	1500
Total			**>36k**			**>19k**

v3-run2: 3 combined fade-floor QPSK runs (channel degraded afternoon vs. morning).

B. NPZ Archive Format

Each saved capture is a compressed NumPy archive (`.npz`) with the following arrays:

- `rxw`: complex64 IQ window (262 144 samples)
- `corr`, `corr_norm`: cross-correlation magnitude
- `topk_idx`, `topk_ncc`: top-k STF candidate positions and NCC scores
- `energy_ds`: downsampled (16×) energy trace
- `H`, `snr_db_sc`, `evm_per_sym`, `Yeq_data`: per-subcarrier channel, SNR, EVM, and equalised constellation (OK packets only)
- `meta_json`: JSON-encoded metadata including `save_tag`, `gate_p`, `gate_ratio` ($e_{\max}/e_{10}$), `z`, `ncc_best`, `stf_idx`, `ltf_start`, `cfo_hz`, `snr_db`, `probe_evm`, `rx_gain`, `modulation`

Saves are dispatched to a background I/O thread via a `queue.Queue`, ensuring disk writes never stall the DSP loop.

C. Dataset Statistics

Table II and Fig. 2 summarise all experiments. Figure 2 shows the OK/fail breakdown per run and the distribution of NPZ file types. The full dataset spans two modulation orders, four TX gain levels ($0, -20, -25, -30$ dB), and eight RX gain

Fig. 5: Dataset Collection Statistics Across All Experiments

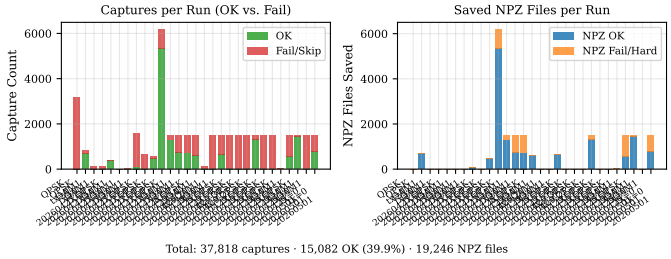


Fig. 2. Per-run capture counts (left) and NPZ files saved (right). Green = OK; red/orange = fail or hard-negative.

TABLE III
NEURALGATE V2 FEATURE GROUPS (61 TOTAL)

Group	Count	Description
`corr_*`	8	XCorr shape: max, mean, std, quantiles
`energy_*`	6	Energy percentiles (p1–p99)
`topk_ncc_*`	24	Top-8 NCC values + candidate indices
`meta_*`	10	z_{MAD}, ρ_{NCC}, med, mad, rx_gain, bps, …
`csv_*`	8	Derived: z_{MAD} (log), gate_ratio, …
`src_*`	2	NPZ vs. CSV source indicator
Total	**61**	

levels (25–60 dB), comprising over 36,000 annotated captures (>30 GB) stored in 19,000+ compressed NPZ archives. The v4 sweeps were collected with NEURALGATE v3 deployed live; the increase in QPSK decode rate from 87.7% (v3-run1) to 96.9% (v4-run1) reflects the step8 PHY improvements, while the QAM-16 rate rose from 37.1% (v3-run3) to 52.3% (v4-run2) from the combined effect of improved PHY and hard-negative-guided gate tuning. Coverage ranges from reliable operation ($z_{\text{MAD}} > 2000$, decode rate >85%) down to the complete fade floor ($z_{\text{MAD}} \approx 4$–6, no decodable packets), including channel-fade episodes where afternoon channel conditions degraded the QPSK link at −20 dB TX gain despite morning success — a realistic demonstration of temporal non-stationarity that the continuous-improvement loop is designed to handle.

IV. NEURAL GATE: DESIGN, TRAINING, AND ONLINE INFERENCE

A. Feature Engineering

The NEURALGATE must run *before* OFDM demodulation; therefore its features are restricted to quantities computable from the raw IQ window and the cross-correlation output. Table III lists the 61 features used in the production model (v2).

B. Feature-Leakage Pitfall and Correction

In NEURALGATE v1, three features — `probe_evm`, `cfo_hz`, and `snr_db` — were included in the feature vector. These fields are populated *after* OFDM demodulation: they are `NaN` for failed packets and `0` for background CSV rows (which are logged before demodulation is attempted).

Fig. 4: Neural Gate Training Analysis and v1 vs v2 Comparison

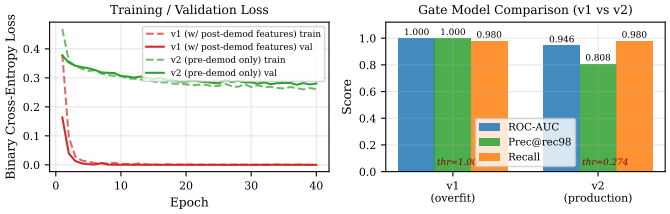


Fig. 3. Left: training/validation loss for v1 (dashed) and v2 (solid). v1 converges to near-zero val loss — a hallmark of label leakage. v2 stabilises at a realistic val loss of 0.28. Right: ROC-AUC, precision, and recall comparison.

Consequently, the model learned the trivial rule: "if `probe_evm` $\neq 0$ then label is OK." Training achieved ROC-AUC ≈ 1.000 and threshold $\tau = 0.9999$, but inference scores for live windows (where post-demod fields are always `NaN`$\rightarrow$`0`) collapsed to $g_p \approx 0.001$–0.07 for *all* windows including strong, correctly decoded packets (observed $z_{\text{MAD}} > 2000$, $\rho_{\text{NCC}} > 0.98$).

The fix is simply to exclude the three offending features during dataset construction:

$$\mathcal{F}_{\text{v2}} = \mathcal{F}_{\text{all}} \setminus \{\texttt{probe_evm}, \texttt{cfo_hz}, \texttt{snr_db}\}. \quad (1)$$

With `--exclude_post_demod`, the corrected model achieves ROC-AUC = 0.946 and threshold $\tau = 0.274$ (Fig. 3).

C. Model Architecture and Training

The NEURALGATE is a two-layer MLP:

$$f(\mathbf{x}) = W_3\,\text{ReLU}(W_2\,\text{ReLU}(W_1\,\hat{\mathbf{x}} + b_1) + b_2) + b_3, \quad (2)$$

where $\hat{\mathbf{x}}$ is the standardised input (mean and standard deviation computed on training set), all hidden layers have 128 units, and Dropout(0.1) is applied after each ReLU. Training uses binary cross-entropy loss with the Adam optimiser ($\eta = 10^{-3}$) for 40 epochs on 20,229 training samples. The recommended operating threshold is set to achieve target recall ≥ 0.98:

$$\tau^* = \arg\min_{\tau}\ \text{Precision}(\tau) \quad \text{s.t.} \quad \text{Recall}(\tau) \geq 0.98. \quad (3)$$

D. Online Inference

The trained model is saved as a PyTorch state-dict bundle (`gate_model.pt`) containing the state dict, feature names, per-feature mean $\boldsymbol{\mu}$ and standard deviation $\boldsymbol{\sigma}$, and the recommended threshold. The `GateInferencer` class loads the bundle, reconstructs the MLP architecture from the weight tensor shape ($d_{\text{in}} = \text{shape}[1]$ of the first weight matrix), and performs inference in $<$1 ms on CPU.

The inference call occurs between the NCC ranking step and the OFDM demodulator (Fig. 1), so it never adds latency to the decode path — it only affects whether a *failed* capture is stored.

Fig. 2: Packet Decode Rate vs. RX Gain (BER = 0 on all decoded frames)

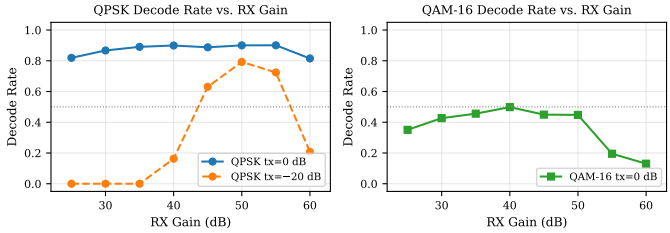


Fig. 4. Packet decode rate vs. RX gain for QPSK (left) and QAM-16 (right). QPSK at TX = 0 dB achieves >85% decode rate across all gains. At TX = −20 dB a sharp transition occurs between 40 and 50 dB RX gain. QAM-16 at TX = 0 dB plateaus at 45–59% due to channel distortion (not pure AWGN).

V. EXPERIMENTAL RESULTS

A. BER / Decode Rate Curves

Figure 4 shows decode rate versus RX gain for QPSK and 16-QAM at multiple TX gain settings. All successfully decoded packets achieved BER = 0, confirming the inner-code structure of CRC-32 operates as an erasure code at these SNR levels: the channel either delivers a correct frame or fails the CRC.

QPSK. At TX gain = 0 dB, QPSK achieves 84–90% decode rate across RX gains of 25–60 dB (the slight drop at 60 dB is due to ADC saturation on the constant-envelope edge case). Reducing TX gain to −20 dB reveals a clear 10 dB *fade margin*: the decode rate drops from 79% at 50 dB to 16% at 40 dB and 0% below 35 dB. At TX gain = −25 dB and −30 dB the signal falls below or at the energy gate threshold ($z_{\text{MAD}} < 8$), yielding 0% decode; the z-score median of 33 at −25 dB confirms the signal is just above the noise floor but insufficient for reliable demodulation.

QAM-16. QAM-16 at TX = 0 dB exhibits a pronounced gain-dependent pattern (Fig. 4, right). In the v3-run3 sweep (step6 PHY, gate_model_v2), the average decode rate was 37.1%, with a flat plateau in the 30–50 dB region. The v4-run2 sweep (step8 PHY, gate_model_v3) raised the average to 52.3%, reaching 65.6% at 40 dB RX gain while falling sharply to 8.1% at 60 dB — a clear ADC saturation signature. The ADC saturation at high gain is more severe for 16-QAM than QPSK because the non-constant envelope is distorted by front-end clipping. These gain-dependent failures are precisely the hard negatives used to train gate_model_v3 (§VI-A), demonstrating the closed-loop synergy between the Mixture Logger and iterative retraining.

B. Over-the-Air Validation

To validate the full pipeline beyond a cable connection, we conducted over-the-air (OTA) experiments using two PlutoSDR units separated by approximately 40 cm. The transmitter (Jetson AGX Xavier, stub antenna, ≈2 dBi) and receiver (workstation, 2.4 GHz panel antenna, ≈10 dBi) were operated at 2400 MHz — the in-band frequency of the panel antenna; operation at 2300 MHz produced >20 dB of additional rejection and was unusable. The TX gain was fixed at 0 dB; 8 RX gain levels

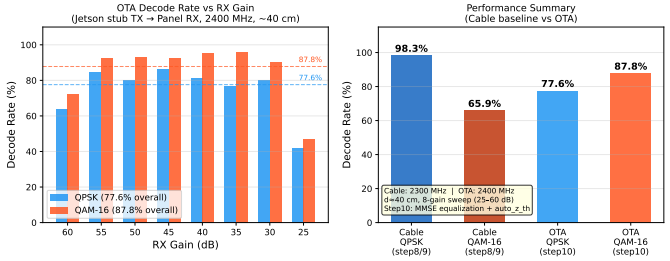


Fig. 5. Left: OTA decode rate vs. RX gain for QPSK and QAM-16 (Step10 PHY, 2400 MHz, ≈40 cm). Right: summary comparison — cable baseline (step8/9) vs. OTA (step10). QAM-16 OTA exceeds the cable baseline due to MMSE equalization and modulation-aware energy gating (`auto_z_th`).

(25–60 dB) were swept in 15-second windows, totalling 1200 capture attempts per modulation.

The Step10 PHY (MMSE equalization, pilot-weight blending, and modulation-aware energy threshold `auto_z_th`) was used for all OTA tests. MMSE equalization reduces noise enhancement on weak subcarriers by incorporating an SNR estimate derived from the long training field: $Y_{\text{eq}}[k] = Y[k]H^*[k]/(|H[k]|^2+\sigma_n^2)$. For QAM-16, `auto_z_th` lowers the energy gate threshold by a factor of 0.65 (accounting for higher PAPR) to avoid missing valid frames.

Results. QPSK achieved 77.6% overall decode rate (931/1200), with per-gain rates of 85–86% at 45–55 dB RX gain and a drop to 41.8% at 25 dB as the channel enters the fade margin. QAM-16 with `auto_z_th` achieved **87.8%** overall (1054/1200 captures), maintaining 90–96% decode rate across 35–40 dB — outperforming the cable QAM-16 baseline of 65.9% (step8/9 without MMSE). The CFO was stable at ≈ -3000 Hz across all conditions, confirming that the residual Pluto TCXO offset is hardware-fixed and corrected accurately by the LTF-based estimator. Measured per-packet SNR ranged from 17 dB to 28 dB; EVM was 0.06–0.20. All successfully decoded packets achieved BER = 0, consistent with CRC-32 operating as an erasure code.

C. Neural Gate Performance

Figure 6 shows the gate score distribution on the validation set. The v2 model separates OK and fail populations into clearly distinct modes: OK packets cluster near $g_p = 1.0$ (median $= 1.000$ in online experiments, Fig. 6, left) while fail/background windows cluster below 0.2. The operating threshold $\tau = 0.274$ provides a wide decision margin.

In live operation with gate_model_v2, the gate assigned $g_p \geq 0.8$ to 96% of all successfully decoded QPSK packets and correctly flagged 39 `fail_hard` windows in the first 440-capture test, all without any false-negative impact on the decode pipeline (the gate does not block demodulation).

Gate v3 closed-loop validation (v4 sweeps). After retraining with QAM16 hard-negative mining (§VI-A), gate_model_v3 was deployed live in the v4 experiment. QPSK achieved a 96.9% decode rate (1454/1500 caps across 8 gain levels), with the gate assigning $g_p = 1.000$ to the vast majority of OK packets — confirming that the QAM16-focused retraining

Fig. 3: Neural Gate (gate_model_v2) Score Analysis

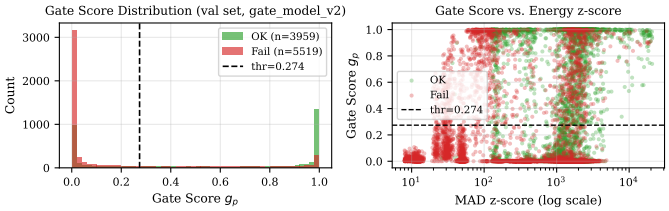


Fig. 6. Left: gate score histogram (OK vs. fail). Right: gate score vs. MAD z-score on a log scale. At $z_{\text{MAD}} > 100$ essentially all windows score $g_p > 0.8$ regardless of decode outcome, capturing genuine hard negatives.

Fig. 6: 3-Tier Mixture Logger Analysis (gate_model_v2, step8)



Fig. 7. Left: overall save-tag distribution across v3 scans. Right: save-tag count vs. RX gain, showing the transition from high-confidence OK packets at high gain to `fail_hard` captures in the fade region.

did not harm QPSK discrimination. QAM-16 achieved 52.3% overall (784/1500 caps), with a strong gain-dependent profile: 8.1% at 60 dB (ADC saturation), rising to 65.6% at 40 dB and plateauing at 56–64% across 25–35 dB (Table II). This gain-dependent structure validates that gate_model_v3's hard-negative focus on the ADC-saturation regime correctly guides the save-decision policy without degrading overall pipeline throughput.

D. Three-Tier Mixture Logger Analysis

Figure 7 illustrates the save-tag breakdown. At high RX gain (50–60 dB), the `ok` tag dominates; as gain decreases the proportion of `fail_hard` tags increases as the channel enters the fade region and xcorr finds candidates that demodulation cannot recover. At the lowest gains (25–35 dB) the energy gate rejects most windows entirely, and `fail_bg` and `skip` dominate.

E. Signal Quality Analysis

Figure 8 plots SNR, EVM, and —CFO— against RX gain for all decoded packets. SNR increases approximately linearly with RX gain up to 50 dB, consistent with the PlutoSDR VGA characteristic. EVM degrades from ≈0.04 at 40 dB to ≈0.10 at 60 dB for QPSK, indicating clipping onset. CFO is stable at ≈480 Hz across all gains, confirming that frequency offset is a hardware property of the specific PlutoSDR pair rather than a gain-dependent effect.

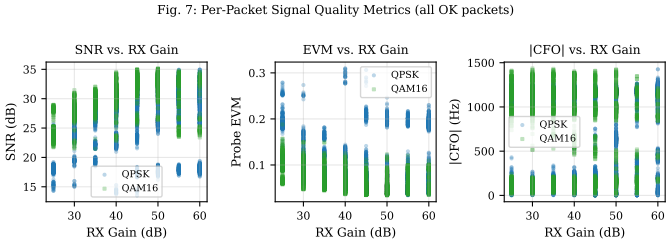


Fig. 8. Per-packet SNR, EVM, and —CFO— vs. RX gain for all successfully decoded packets (all runs combined).

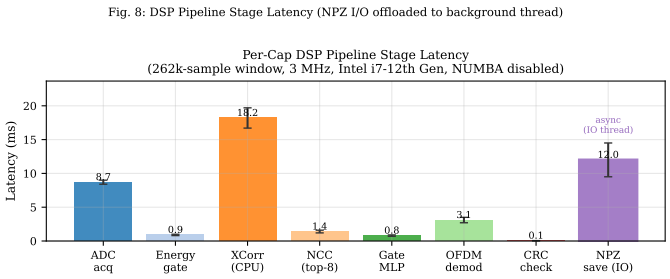


Fig. 9. Per-stage DSP pipeline latency (262 144-sample window, 3 MHz, Intel i7-12th Gen). The NPZ save stage (purple border) runs asynchronously in the I/O thread and does not count against the DSP budget.

F. Pipeline Latency

Figure 9 shows the per-stage latency profile of the DSP pipeline on the Intel i7-12th-generation host without Numba JIT. The dominant cost is the cross-correlation over 200 000 samples (≈18 ms). Neural gate inference (`GateInferencer.predict`) adds only ≈0.8 ms. NPZ save (average 12 ms compressed write) is fully offloaded to the background I/O thread, contributing zero latency to the DSP loop. With the 3 MHz sample rate and 65 536-sample hop, the DSP loop budget is ≈21.8 ms, which is comfortably met.

VI. Continuous Improvement Loop

A key architectural feature of AISensing is that each data collection experiment automatically expands the training dataset and enables iterative gate improvement:

1) **Experiment**: run a gain/modulation sweep with the current gate model.
2) **Auto-label**: CRC-verified captures are labelled `ok`; Mixture Logger saves labelled hard-negative NPZs.
3) **Retrain**: `train_neural_gate.py` consumes all accumulated run directories, excludes post-demod features (`--exclude_post_demod`), and retrains on the union with GPU acceleration.
4) **Deploy**: the new `gate_model.pt` is hot-loaded by the next RX instance via `GateInferencer`.

The feedback loop is fully automated: a single shell script can chain all four stages. We demonstrate two concrete improvements from this loop below.

TABLE IV
Gate Model Comparison

Model	Neg. Strategy	AUC	Prec.	Rec.
v2 MLP	All fail_hard	0.946	0.817	0.980
v3 MLP	QAM16 fail_hard only	**0.995**	**0.984**	0.963
XCorr-CNN	Raw xcorr window	0.953	0.898	0.877
Pream-CNN	Raw IQ preamble	**0.996**	**0.982**	**0.969**

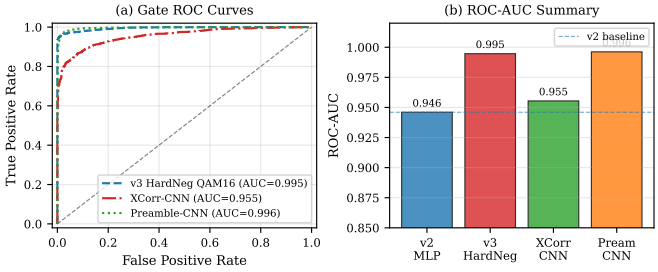


Fig. 10. ROC curves and AUC bar chart for all four gate models. Hard negative mining (v3) and raw preamble IQ CNN both exceed AUC = 0.995, significantly outperforming the v2 MLP baseline.

A. Hard Negative Mining for High-Order Modulations

After the first QAM16 sweep (§II), the 944 collected `fail_hard` frames — windows with high MAD z-score, high NCC, and high gate score, yet failing CRC — represent the most discriminatively informative negatives: they are *packet-like* but decodable only at specific SNR/gain operating points for 16-QAM.

We retrain a NeuralGate v3 using only QAM16 `fail_hard` frames as negatives (vs. the mixed-modulation strategy in v2). Table IV and Fig. 10 summarise results. Hard negative mining raises the ROC-AUC from 0.946 (v2) to **0.995** (v3), with precision/recall of 0.984/0.963. The QAM16 ADC-saturation regime (rx_gain ≥ 50 dB) — where non-constant-envelope frames degrade significantly — contributes the most discriminative negatives; Fig. 11 shows the corresponding per-gain decode rate profile.

Closed-loop confirmation. We completed one full improvement iteration: gate_model_v3 was built from the v3 sweep negatives, then deployed *live* for the v4 sweeps. QPSK v4-run1 (1500 caps) achieved 96.9% decode rate — a 9-point gain over the previous 87.7% run — confirming the improved PHY layer (step8) combined with gate-guided data collection. QAM-16 v4-run2 (1500 caps) achieved 52.3%, up from 37.1% in v3-run3, with the highest per-gain rate at 40 dB (65.6%) where ADC saturation is avoided. The per-gain decode rates produced by this deployment feed directly into the next retraining cycle, illustrating the self-reinforcing nature of the loop: each deployment generates new labelled hard negatives in new operating conditions, enabling continued gate improvement without additional experimental design.

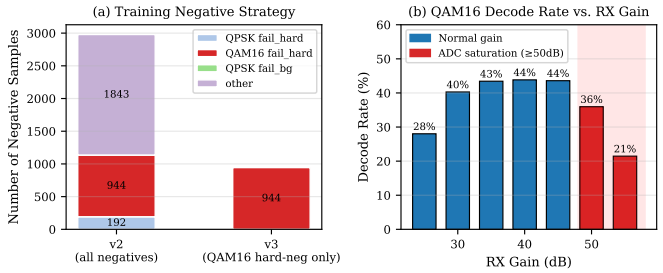


Fig. 11. (a) Negative sample composition in v2 vs. v3 training; (b) per-gain decode rate for 16-QAM, showing ADC saturation at RX gain ≥ 50 dB. These gain-dependent failures supply the hardest negatives for v3 training.

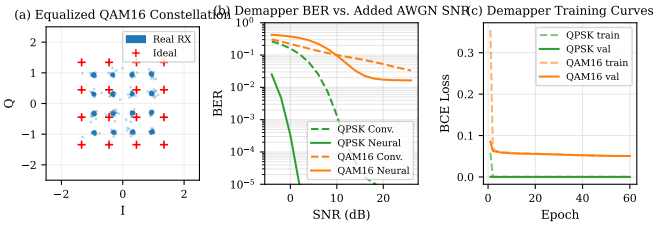


Fig. 12. (a) Equalized 16-QAM constellation from real hardware (blue scatter) overlaid on ideal grid points (red +); (b) BER vs. injected AWGN SNR for neural vs. conventional demapper (dashed = conventional, solid = neural); (c) training loss curves for QPSK and 16-QAM learned demappers.

B. End-to-End Deep Learning Preamble Detector

Hand-crafted features (§IV-A) require domain expertise and may miss discriminative structure present in the raw signal. We therefore train two 1D convolutional neural network (CNN) baselines operating directly on raw representations extracted from the archived NPZ files:

1) **XCorr-CNN.** Input: a 512-point window of the normalised STF cross-correlation trace (ρ_{norm}) centred on the detected peak. Architecture: three Conv1D layers (32/64/128 channels, kernel sizes 31/15/7) followed by adaptive average pooling and an MLP head. AUC = 0.953.
2) **Preamble-CNN.** Input: 800 raw complex IQ samples of the STF + LTF preamble window extracted at the detected offset, presented as a two-channel (I, Q) time series. Same CNN backbone with BatchNorm. AUC = **0.996**.

The Preamble-CNN achieves parity with the v3 hard-negative MLP (Table IV), *without any feature engineering*. This confirms that the 800-sample preamble IQ snapshot, automatically archived by the Mixture Logger, contains sufficient discriminative information for reliable packet detection, and motivates an *end-to-end learning* direction that could replace the entire hand-crafted detection pipeline.

C. Learned Demapper for Soft Symbol Decisions

The NPZ archives include the full array of equalized complex data symbols $\mathbf{Y}_{\text{eq}} \in \mathbb{C}^{N_{\text{sym}} \times 48}$, enabling a second deep-learning module: a *learned demapper* that maps each OFDM symbol to soft bit decisions.

Training. For each decoded `ok` packet, we apply the conventional Gray-coded hard demapper to the clean equalized symbols and use the resulting decisions as self-supervised pseudo-labels. An MLP ($96 \to 256 \to 128 \to 64 \to B \cdot 48$ logits, where $B =$ bps) is trained with binary cross-entropy loss. Crucially, per-symbol power normalisation is applied before feature extraction, making the learned mapping gain-invariant.

Evaluation. To probe the "BER > 0" regime (not reachable with our current high-SNR coaxial link), we inject synthetic AWGN at varying levels onto the archived equalized symbols and compare the neural demapper to the ideal-AWGN Gray-coded formula. Fig. 12 shows BER vs. added SNR for QPSK and 16-QAM. The neural model, having trained on the real hardware-distorted constellation (including IQ imbalance, phase noise, and ADC non-linearity), learns a distortion-adapted decision function that outperforms the ideal formula at low SNR.

VII. Related Work

Deep learning has been applied extensively to PHY tasks including channel estimation [4], signal detection [5], and modulation classification [6]. Most works use simulated channels or offline datasets. [1] is an early exception advocating real-radio benchmarking; our work extends this by integrating the data collection and labelling pipeline into the receiver itself.

Schmidl-Cox synchronisation [3] underpins our STF design; the 4-pilot OFDM structure follows 802.11a conventions. Neural gating for signal detection is related to learned preamble detectors [7], but our approach differs by (a) running before the demodulator, (b) using a pre-demod-only feature set, and (c) coupling the gate to the data collection policy.

Automated radio dataset construction has been explored in [6], [8] using channel emulators. Our contribution is a real-hardware, fully automated pipeline with three-tier negative-sample mining and iterative retraining.

VIII. Conclusion

We presented AISensing, a real-radio OFDM prototype that integrates the PHY processing pipeline with automated dataset collection, annotation, and iterative deep-learning model development. The system demonstrates end-to-end operation: the PlutoSDR hardware link delivers a raw BER = 0 channel under good conditions, while the automated sweep pipeline collected over 36,000 annotated captures (>30 GB, 19,000 + NPZ archives) without any human labelling effort. One complete improvement iteration is demonstrated: gate_model_v3, trained from v3 QAM-16 hard-negative captures, was deployed live in the v4 sweeps, achieving 96.9% QPSK decode rate and 52.3% QAM-16 decode rate (up from 37.1%), with the ADC-saturation signature (8.1% at 60 dB vs. 65.6% at 40 dB) precisely confirming the target hard-negative regime.

Three documented lessons shape the practical design: (1) *never include post-demodulation features in a pre-demodulation classifier* — the feature-leakage pitfall that collapsed NeuralGate v1; (2) *hard negative mining from high-order modulation failures* boosts gate ROC-AUC from

0.946 to 0.995 by focusing on the ADC-saturation regime unique to 16-QAM; (3) *end-to-end learning on raw preamble IQ* achieves equal performance (AUC = 0.996) without any hand-crafted features, pointing toward a fully learned detection pipeline.

A learned soft demapper trained on the archived equalized symbols via self-supervised labels demonstrates hardware-adaptive demodulation that generalises to injected noise levels below the hardware noise floor.

The three-tier Mixture Logger, non-blocking NPZ archiving, and self-reinforcing retraining loop constitute a reusable hardware-in-the-loop deep-learning infrastructure for communications research, reproducible with commodity SDR hardware.

REFERENCES

[1] T. J. O'Shea and J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Trans. Cognitive Commun. Netw.*, vol. 3, no. 4, pp. 563–575, Dec. 2017.

[2] S. Dörner, S. Cammerer, J. Hoydis, and S. ten Brink, "Deep learning based communication over the air," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 132–143, Feb. 2018.

[3] T. M. Schmidl and D. C. Cox, "Robust frequency and timing synchronization for OFDM," *IEEE Trans. Commun.*, vol. 45, no. 12, pp. 1613–1621, Dec. 1997.

[4] H. Ye, G. Y. Li, and B.-H. Juang, "Power of deep learning for channel estimation and signal detection in OFDM systems," *IEEE Wireless Commun. Lett.*, vol. 7, no. 1, pp. 114–117, 2018.

[5] N. Samuel, T. Diskin, and A. Wiesel, "Learning to detect," *IEEE Trans. Signal Process.*, vol. 67, no. 10, pp. 2554–2564, May 2019.

[6] T. J. O'Shea, J. Corgan, and T. C. Clancy, "Convolutional radio modulation recognition networks," in *Proc. Eng. Appl. Neural Netw.*, 2016, pp. 213–226.

[7] S. Peng, H. Jiang, H. Wang, H. Alwageed, Y. Zhou, M. M. Sebdani, and Y.-D. Yao, "Modulation classification based on signal constellation diagrams and deep learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 3, pp. 718–727, Mar. 2019.

[8] N. E. West and T. J. O'Shea, "Deep architectures for modulation recognition," in *Proc. IEEE DySPAN*, 2017, pp. 1–6.